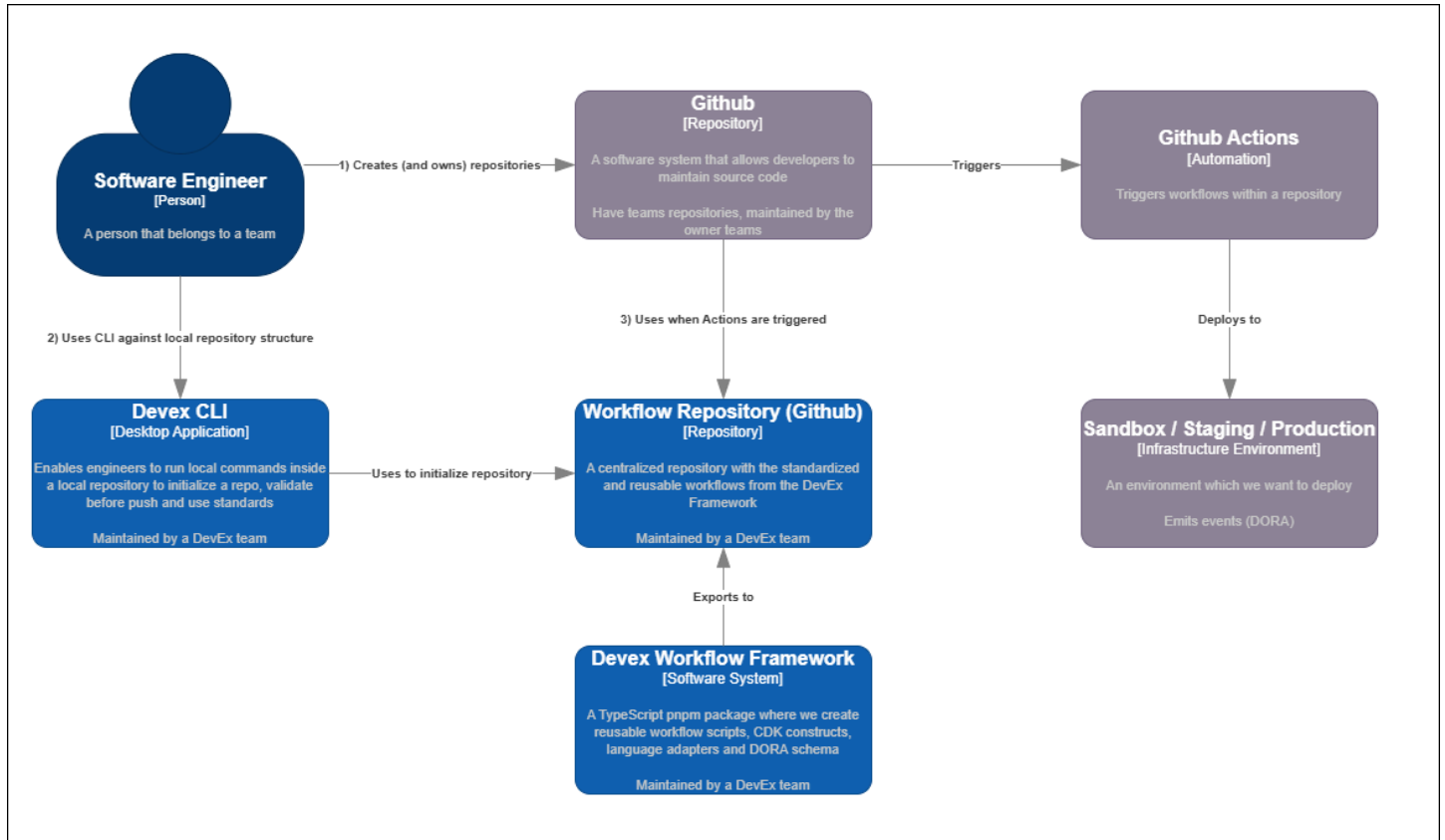


Architecture Diagram



Homologation

The CLI acts as the entry point for developers, while the TypeScript Framework provides the reusable CI/CD, infrastructure, governance, and telemetry logic behind the scenes.

Instead of asking each team to manually copy standards into their own repositories, the CLI initializes a repository with the required baseline: devex.yaml, GitHub Actions caller workflows, PR templates, Git hooks, Amazon Q rules, and Kiro steering files (and anything else if needed, can be updated through time).

The standardization model is based on convention over configuration. Teams are not blocked from customizing their service workflows, but customization happens through controlled extension points rather than by forking the Golden Path.

To drive adoption across 10+ teams, the platform team would provide a simple onboarding path:

- 1) Install the CLI using uv
- 2) Run devex init inside the service repository
- 3) Commit the generated configuration, workflow caller, PR template, hooks, and AI review context
- 4) Use devex branch, devex validate, and devex pr during normal development
- 5) Let the shared TypeScript Framework execute the standardized PR and integration pipelines

Scalability

The platform avoids becoming a bottleneck by separating the Golden Path into a stable core and well-defined extension points. The stable core contains the mandatory standards: Work ID enforcement, PR governance, required test stages, CDK deployment stages, DORA telemetry, audit logging, and shared infrastructure

patterns. These rules remain centralized in the TypeScript Framework so that all teams continue to produce comparable lifecycle data.

Custom behavior is handled through configuration, adapters, and inner-source contributions rather than one-off workflow forks. The main customization surface is `devex.yaml`, where each team can define its language runtime, test commands, contract test command, CDK working directory, stack names, deployment environments, and telemetry sink. This allows teams to adapt the framework to their service without changing the reusable workflow itself.

[Code] Language-specific needs are handled through framework adapters. The TypeScript Framework can expose a `LanguageAdapter` interface for Python, Go, TypeScript, .Net, Java and future languages. Each adapter knows how to install dependencies, run unit tests, run property-based tests, and execute contract validations for that ecosystem. When a team needs support for a new language or testing strategy, they can contribute a new adapter instead of asking the platform team to manually customize their pipeline.

[Infrastructure] The same principle applies to infrastructure. Shared CDK constructs such as `GoldenLambdaApi` provide recommended infrastructure patterns, but teams can contribute new constructs for other approved patterns. These contributions follow inner-source guidelines, including tests, documentation, examples, and compatibility expectations.

Shift-Left Strategy

The CLI is the primary shift-left mechanism. It gives developers local commands that mirror the checks executed later in Github Actions. For example, `Devex CLI check` validates repository standards, `Devex CLI validate` runs mandatory local checks, `Devex CLI branch` creates compliant branch names, and `Devex CLI pr` creates PRs using the expected Work ID and template format. This helps developers follow the rules before opening a pull request.

Git hooks provide another local enforcement layer. During initialization, the CLI installs a `commit-msg` hook to ensure every commit includes the required Work ID, and a `pre-push` hook to run mandatory validation before code reaches the remote repository. The `pre-push` hook can run linting, unit tests, property-based tests, contract tests, and repository standards checks using the same commands declared in `devex.yaml`.

The PR pipeline then acts as a fast feedback loop. It validates the same conventions remotely, runs the Small Tests stage, performs CDK synthesis, and executes the deployment promotion flow. Because these stages are standardized in the TypeScript Framework, every team receives consistent feedback in the same format, regardless of whether the service is written in Python, Go, .Net, Java or TypeScript.

AI-assisted review is also shifted left by including Amazon Q (or Github Copilot Review) review context and repository-specific rules during initialization. The CLI creates (for example) `.amazonq/rules` files that describe the expected engineering standards, including Work ID (like a JIRA Card Id) usage, audit logging, testing expectations, CDK patterns, and security considerations. This allows automated review to check the change against the same Golden Path expectations used by humans and CI/CD.

Kiro steering files support the same strategy by giving AI-assisted development tools persistent context about the service architecture, platform standards, testing model, and delivery workflow. This reduces the likelihood that generated code or suggested changes drift away from the organization's engineering conventions.